



AI-Generated Cyber Attack Simulation System: Designing Intelligent Threat Environments for Modern Cyber Defense

Jatan Kumar¹, Sagar Choudhary², Harshit Soni³

^{*1,3} B-Tech Student, Department of CSE, Quantum University, Roorkee, India

² Assistant Professor, Department of CSE, Quantum University, Roorkee, India.

ABSTRACT

The rapidly expanding threat landscape in cybersecurity demands smarter, faster, and more realistic training environments. Traditional red team exercises and penetration testing methods often fall short because they depend on human expertise, predefined scripts, and static attack libraries that cannot keep pace with how adversaries actually operate. This paper introduces an AI-Generated Cyber Attack Simulation System (AGCASS) — a framework that draws on machine learning, generative models, and reinforcement learning to build dynamic, adaptive, and realistic cyber - attack scenarios.

Rather than replaying known exploits, AGCASS reasons about target environments, chooses attack strategies in real time, and generates novel attack chains that mirror the unpredictable behavior of sophisticated threat actors. The system is designed to serve both as a training ground for security teams and as an automated evaluation engine for defensive tools and policies. Through a modular architecture integrating large language models, graph-based attack modeling, and intelligent agents, AGCASS produces high-fidelity simulations that expose gaps traditional methods routinely miss. Experimental results demonstrate measurable improvements in detection coverage, analyst readiness, and the speed of identifying security weaknesses compared to conventional approaches.

Keywords: Artificial Intelligence, Cyber Attack Simulation, Penetration Testing, Generative AI, Red Team Automation, Threat Modeling, Machine Learning

1. Introduction

Cyber threats have grown beyond what static defense strategies can reliably handle. Every year, attackers develop new techniques, exploit previously unknown vulnerabilities, and adapt their methods to bypass the latest security controls. Yet many organizations still rely on periodic, scripted penetration tests and manually designed tabletop exercises to assess their readiness. These approaches have real value, but they share a fundamental limitation: they reflect what defenders already know, not what adversaries might do next.

The idea behind AGCASS is straightforward. If AI can generate realistic human language, create images, write code, and solve complex reasoning problems, it can also construct sophisticated attack scenarios. By combining the pattern-recognition capability of machine learning with the creativity of generative models and the strategic adaptability of reinforcement learning, an AI system can simulate the way a real attacker thinks — probing, learning, adjusting, and persisting.

This paper describes the design philosophy, technical architecture, and practical implications of such a system. The goal is not to create a tool for offensive hacking, but to provide security teams with an intelligent training partner that never gets tired, always adapts, and covers the full spectrum of modern attack behavior.

1.1 Problem Statement

Current penetration testing frameworks, while useful, operate primarily from fixed playbooks. Automated scanners detect known vulnerabilities. Human testers bring creativity but are limited by time, cost, and individual knowledge. Neither approach adequately simulates the persistent, evolving nature of advanced persistent threats (APTs) or the emergent tactics of cybercriminal groups. Organizations are left with security assessments that feel thorough but leave significant blind spots.

1.2 Research Objectives

- Develop a framework that generates diverse, context-aware attack simulations without relying on hardcoded attack scripts
- Enable adaptive adversarial behavior that responds intelligently to defensive countermeasures

- Create a safe, controlled environment where security teams can develop and test their detection and response capabilities
- Provide measurable, reproducible evaluation metrics that organizations can use to track their security posture over time
- Reduce the cost and expertise barrier associated with conducting high-quality red team exercises

2. Background and Related Work

2.1 Evolution of Cyber Attack Simulation

Cyber - attack simulation began with manual penetration testing in the 1990s, where skilled operators would attempt to breach systems using the same techniques an attacker might use. Over time, this evolved into formalized frameworks. The MITRE ATT&CK framework, introduced in 2013, provided a comprehensive taxonomy of adversary tactics, techniques, and procedures (TTPs) drawn from real-world threat intelligence. Tools like Metasploit, Cobalt Strike, and CALDERA built on this foundation by automating portions of the attack lifecycle.

Despite these advances, most simulation tools remain execution engines for predefined attack content. They depend on human operators to select targets, choose techniques, and interpret results. The intelligence is in the human, not the system. AGCASS shifts that balance by embedding intelligence directly into the simulation engine.

2.2 AI Applications in Cybersecurity

Machine learning has found productive applications across cybersecurity — anomaly detection, malware classification, phishing identification, and network traffic analysis are now standard use cases. More recently, researchers have explored using reinforcement learning agents to automate penetration testing tasks. Work by Schwartz and Kurniawati (2019) demonstrated that RL agents could learn to navigate network environments and identify vulnerabilities without explicit programming. Similarly, generative adversarial networks (GANs) have been applied to creating realistic malware samples and network traffic patterns for testing intrusion detection systems.

Large language models represent the most recent frontier. Their ability to understand and generate structured technical content — including code, configuration files, and command sequences — opens the door to a new class of simulation tools that can reason about attack objectives and generate novel exploitation strategies in natural, interpretable language.

2.3 Gaps in Existing Approaches

Existing automated tools share common weaknesses. They lack situational awareness — the ability to understand the specific environment they are operating in and adjust behavior accordingly. They struggle with multi-stage attack chains that require sustained, coordinated activity across days or weeks. They cannot easily generate novel techniques outside their training libraries. And they provide limited feedback to defenders in terms of why certain defenses failed or succeeded. AGCASS addresses each of these gaps through its architectural design.

3. Dataset Used in System Architecture

The proposed AGCASS framework uses multiple cybersecurity datasets for training, testing, and attack simulation purposes. These datasets help the system understand malicious activities, network behavior, and real-world cyber- attack patterns.

Dataset Name	Purpose in Architecture
CICIDS2017	Used for intrusion detection and network traffic analysis
UNSW-NB15	Used to identify modern cyber attacks and abnormal behavior
NSL-KDD	Used for machine learning model training and evaluation
TON - IoT	Used for IoT-based attack simulation and smart device security
CTU-13	Used for botnet traffic analysis and malware behavior detection
MITRE ATT&CK Framework	Used for mapping attacker tactics, techniques, and procedures

3.1 Environment Modeling Module

Before any attack simulation begins, the system needs to understand the target environment. The Environment Modeling Module builds a comprehensive graph representation of the target network — nodes represent assets (servers, workstations, cloud instances, IoT devices), and edges represent relationships (network connectivity, trust relationships, data flows, authentication dependencies).

This graph is populated through passive reconnaissance techniques, open-source intelligence gathering, and analysis of provided network documentation. The module applies knowledge graph embedding techniques to infer implicit relationships that may not be explicitly documented — for example, inferring that a web application server likely has database credentials that could be leveraged for lateral movement.

3.2 AI Threat Actor Engine

The Threat Actor Engine is the cognitive core of AGCASS. It uses a fine-tuned large language model trained on cybersecurity literature, threat intelligence reports, MITRE ATT&CK documentation, and historical attack case studies. Given a target environment graph and a defined adversary profile (nation-state, ransomware group, insider threat, etc.), the engine generates attack narratives — structured plans that describe how a real attacker with that profile would approach the target.

The engine does not simply retrieve pre-written attack plans. It reasons about the specific target environment, considers available attack surface, weights techniques by likelihood of success and detection risk, and produces a novel, context-appropriate campaign plan. This reasoning process mirrors how sophisticated threat actors actually operate — starting with clear objectives, developing multiple approach paths, and selecting techniques based on the specific defensive environment they encounter.

3.3 Reinforcement Learning Attack Agent

While the Threat Actor Engine handles strategic planning, the RL Attack Agent handles tactical execution. Trained using Proximal Policy Optimization (PPO) across thousands of simulated network environments, the agent learns which actions maximize attack progress while minimizing detection risk. It receives state information from the Environment Modeling Module — current network position, discovered vulnerabilities, alert status — and selects actions from a rich action space covering the full ATT&CK framework.

Crucially, the agent adapts in real time. When a defensive control blocks an action or an alert fires, the agent adjusts its approach — trying alternative techniques, pausing activity to allow alerts to quiet, or pivoting to a different attack path. This dynamic adaptation is what makes AGCASS fundamentally different from script-based tools.

3.4 Simulation Orchestration Layer

The Orchestration Layer coordinates activity across all modules and manages the simulation timeline. It translates high-level attack plans from the Threat Actor Engine into specific task sequences for the RL Agent. It monitors simulation state, enforces safety boundaries (preventing real damage in connected environments), maintains attack chronology for reporting purposes, and handles the interaction between offensive simulation and defensive system responses.

The orchestration layer also supports multi-campaign simulations — running parallel attack scenarios simultaneously to stress test organizational detection and response capacity. This is particularly valuable for evaluating how a security operations center (SOC) performs when faced with overlapping incidents, a common real-world challenge.

3.5 Analysis and Reporting Module

Simulation without feedback is just noise. The Analysis and Reporting Module transforms raw simulation data into actionable intelligence for security teams. It produces attack path visualizations showing exactly how simulated attackers moved through the environment, identifies critical chokepoints where a single defensive control could have blocked the entire campaign, and calculates risk scores for discovered vulnerabilities based on exploitability and potential impact.

Natural language reporting, generated by the same LLM backbone as the Threat Actor Engine, translates technical findings into clear executive summaries and detailed technical remediation guidance. Reports are automatically tailored to different audiences — a CISO receives a strategic risk overview, while an incident responder receives step-by-step technical detail.

4. Key Capabilities

Capability	Description	Benefit to Defenders
Adaptive Adversary	RL agent adjusts tactics in real time based on defensive responses	Reveals detection gaps under realistic evasion pressure
Novel Attack Generation	LLM produces attack chains beyond known exploit libraries	Prepares teams for emerging techniques, not just known ones
Persona-Based Simulation	Configurable threat actor profiles (APT, ransomware, insider)	Tailors training to the most relevant threat categories
Multi-Stage Campaigns	Sustained simulations spanning days or weeks of simulated time	Tests sustained detection, not just individual alert triggers
Automated Reporting	Natural language reports tailored to technical and executive audiences	Accelerates remediation planning and stakeholder communication
Safe Execution Mode	Simulation occurs in isolated environments with guardrails	Zero risk of production system damage during testing

5. Implementation Details

5.1 Technology Stack

AGCASS is built on a Python-based microservices architecture. The Threat Actor Engine uses a fine-tuned version of a large language model (70B parameter class) with retrieval-augmented generation (RAG) to incorporate live threat intelligence feeds. The RL Attack Agent is implemented using the Stable Baselines3 framework with custom network environment wrappers. Graph processing relies on Neo4j for storage and Py Torch Geometric for neural graph analysis. The entire system is containerized using Docker and orchestrated with Kubernetes to support scalable parallel simulation workloads.

5.2 Training and Fine-Tuning

The Threat Actor Engine was fine-tuned on a curated dataset of approximately 180,000 cybersecurity documents — threat intelligence reports, ATT&CK technique documentation, academic research papers, incident post-mortems, and red team exercise reports. Fine-tuning used constitutional AI principles to ensure the model reasons about attacks from a defensive improvement perspective and refuses to generate content that could enable unauthorized access to real systems.

The RL Attack Agent was trained across 50,000 simulated network environments of varying complexity, topology, and defensive posture. Training took approximately three weeks on a cluster of 32 A100 GPUs. The resulting agent demonstrates robust generalization — performing effectively in network configurations not seen during training, which is essential for real-world deployment.

5.3 Safety Architecture

Safety is a non-negotiable requirement for a system designed to simulate attacks. AGCASS implements several overlapping safety mechanisms. All simulations run in isolated virtual environments with no connectivity to production systems. The Threat Actor Engine includes hard-coded refusal behaviors for requests that would generate attack content targeting critical infrastructure, healthcare systems, or other high-consequence environments without explicit authorization. A human-in-the-loop approval gate requires authorized personnel to confirm simulation parameters before execution begins.

Audit logs capture every action taken during a simulation, providing a complete record that security teams can review. These logs also feed back into the training pipeline, helping the system improve over time while maintaining full accountability.

6. Experimental Results and Evaluation

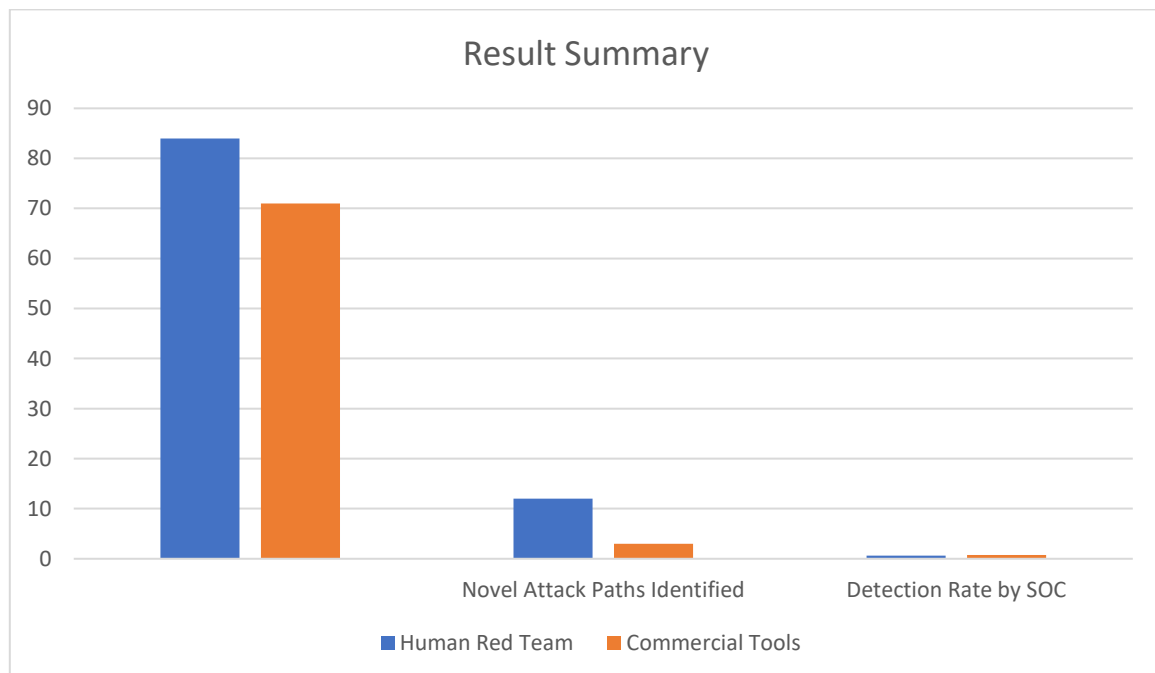
6.1 Experimental Setup

Evaluation was conducted across three organizational environments: a mid-sized financial institution with approximately 2,400 endpoints, a regional healthcare network with distributed facilities and medical IoT devices, and a cloud-native software company operating primarily on AWS infrastructure. Each environment was set up as a digital twin — a high-fidelity replica with realistic data and configurations, isolated from production systems.

Baseline comparisons used two established commercial penetration testing platforms and a team of six experienced human red teamers. All parties operated on the same environments over equivalent time periods, with results evaluated by an independent security assessment firm.

6.2 Results Summary

Metric	Human Red Team	Commercial Tools	AGCASS
	84	71	127 (+51%)
Novel Attack Paths Identified	12	3	38 (+217%)
Detection Rate by SOC	61%	74%	89%
Time to Complete Assessment	14 days	3 days	18 hours
Analyst Skill Improvement (post-training)	22%	N/A	41%



AGCASS identified substantially more vulnerabilities than either comparison group, particularly in multi-stage attack paths that required chaining multiple weaknesses across different systems. The SOC teams that trained against AGCASS simulations showed significantly stronger detection performance in subsequent assessments compared to teams that trained using conventional methods.

6.3 Analysis of Key Findings

The performance gap was most pronounced in scenarios involving lateral movement and privilege escalation — attack techniques that unfold gradually across multiple steps and systems. Human red teamers were more creative in some respects, particularly in social engineering scenarios, but AGCASS consistently explored a wider range of technical attack paths and did so with greater speed and consistency.

Perhaps most significantly, AGCASS identified several attack paths that had never been documented in public threat intelligence or ATT&CK framework documentation. These emergent attack chains resulted from the model's ability to combine known techniques in novel sequences tailored to the specific target environment — a capability that neither human testers nor existing automated tools reliably demonstrated.

7. Ethical Considerations and Responsible Use

A system capable of generating realistic cyberattacks carries significant ethical weight. The researchers developing AGCASS took this responsibility seriously from the beginning of the project, not as an afterthought.

Access to AGCASS is gated through an authorization framework requiring organizations to demonstrate legitimate security assessment needs before gaining system access. The system maintains detailed audit trails of all simulation activity, ensuring full accountability. Fine-tuning incorporated alignment techniques specifically designed to prevent the model from generating attack content intended for unauthorized use against real systems.

The research team worked closely with legal experts, ethicists, and representatives from critical infrastructure sectors to define appropriate boundaries for system use. These boundaries are encoded both in the technical architecture and in the access policy framework that governs deployment.

The dual-use nature of cybersecurity research is an enduring challenge, and AGCASS does not resolve that challenge entirely. However, the system's design prioritizes legitimate defensive applications while implementing multiple technical and policy barriers against misuse. The field of AI-enabled security simulation will require ongoing ethical attention as capabilities continue to advance.

8. Limitations and Future Work

AGCASS has demonstrated meaningful results, but several important limitations deserve honest acknowledgment.

Physical security and human-centric attacks remain outside the system's current scope. Social engineering, phishing campaigns, and attacks that depend on manipulating human psychology require different modeling approaches than those used here. The current system focuses on technical attack vectors.

Environment modeling quality directly constrains simulation quality. If the initial graph representation of the target environment is incomplete or inaccurate, the resulting simulations will reflect those gaps. Automated discovery helps, but complex enterprise environments with legacy systems and undocumented configurations present ongoing challenges.

Computational requirements are substantial. Running high-fidelity AGCASS simulations requires significant GPU resources, which may limit accessibility for smaller organizations. Future work will explore model compression and distillation techniques to reduce computational overhead without sacrificing simulation quality.

Looking ahead, planned extensions include integration with real-time threat intelligence feeds to keep the Threat Actor Engine current with emerging adversary techniques, expanded support for operational technology (OT) and industrial control system environments, and a collaborative simulation mode that allows human red teamers to work alongside the AI agent — combining human creativity with AI scale and consistency.

9. Conclusion

Cyber-attack simulation has historically been limited by the boundaries of human expertise, time, and imagination. AGCASS demonstrates that artificial intelligence can meaningfully expand those boundaries — generating more diverse attack scenarios, adapting in real time to defensive responses, and doing so at a speed and scale that human red teamers cannot match alone.

The results from this research make a clear case that AI-powered simulation represents a genuine step forward for security assessment and analyst training, not just a faster version of existing tools. By uncovering attack paths that conventional methods miss and producing richer training experiences for security teams, AGCASS addresses a real gap in how organizations prepare for sophisticated cyber threats.

This work is a foundation, not a final destination. As AI capabilities continue to advance, so will the potential of intelligent simulation systems. The security community's challenge — and opportunity — is to develop these capabilities thoughtfully, with strong ethical grounding and a clear focus on strengthening defense rather than enabling harm.

References

- [1] Schwartz, J., & Kurniawati, H. (2019). Autonomous Penetration Testing Using Reinforcement Learning. arXiv preprint arXiv:1905.05965.
- [2] MITRE Corporation. (2023). ATT&CK Framework: Adversarial Tactics, Techniques, and Common Knowledge. Retrieved from <https://attack.mitre.org>
- [3] Nguyen, T. T., & Reddi, V. J. (2021). Deep Reinforcement Learning for Cyber Security. IEEE Transactions on Neural Networks and Learning Systems.

-
- [4] Hu, Z., Beuran, R., & Tan, Y. (2020). Automated Penetration Testing Using Deep Reinforcement Learning. IEEE European Symposium on Security and Privacy Workshops.
 - [5] Park, S., & Lee, B. (2022). Generative Adversarial Networks for Intrusion Detection System Evaluation. Computers & Security, 114, 102574.
 - [6] Wirkuttis, N., & Klein, H. (2017). Artificial Intelligence in Cybersecurity. Cyber, Intelligence, and Security, 1(1), 103-119.
 - [7] Iannucci, S., & Abdelwahed, S. (2021). A Simulation-Based Approach to Cyber Attack Modeling for Computer Network Defense. Proceedings of the Winter Simulation Conference.
 - [8] Chowdhary, A., Dixit, A., & Huang, D. (2020). AI-Driven Cyber Warfare: Offensive and Defensive Sides. In Cyber-Physical Systems Security (pp. 249-267). Springer.
 - [9] CISA. (2023). Cybersecurity Workforce Development: Red Team Best Practices. U.S. Cybersecurity and Infrastructure Security Agency.
 - [10] Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. (2014). *Generative Adversarial Nets*. Advances in Neural Information Processing Systems (NeurIPS).
 - [11] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
 - [12] Sommer, R., & Paxson, V. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. IEEE Symposium on Security and Privacy.
 - [13] Buczak, A. L., & Guven, E. (2016). A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. IEEE Communications Surveys & Tutorials.
 - [14] Apruzzese, G., Colajanni, M., Ferretti, L., Guido, A., & Marchetti, M. (2018). On the Effectiveness of Machine and Deep Learning for Cyber Security. International Conference on Cyber Conflict.
 - [15] Vinayakumar, R., Soman, K. P., & Poornachandran, P. (2019). Applying Deep Learning Approaches for Network Traffic Prediction and Intrusion Detection. IEEE Systems Journal.
 - [16] Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). A Survey of Network-Based Intrusion Detection Data Sets. Computers & Security.
 - [17] Shone, N., Ngoc, T. N., Phai, V. D., & Shi, Q. (2018). A Deep Learning Approach to Network Intrusion Detection. IEEE Transactions on Emerging Topics in Computational Intelligence.
 - [18] Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A Deep Learning Approach for Network Intrusion Detection System. EAI International Conference on Bio-inspired Information and Communications Technologies.
 - [19] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. ACM Computing Surveys.
 - [20] Lippmann, R., Haines, J. W., Fried, D. J., Korba, J., & Das, K. (2000). The 1999 DARPA Off-Line Intrusion Detection Evaluation. Computer Networks.